

Sol Quick Reference

Quick reference for ASU's Sol supercomputer - the solx CLI, Slurm job routing, safe automation, storage, and compute-node access.

v1.0.2

JOBS

SCRATCH

SERVICES

01 / KNOW YOUR ACCESS FIRST

Your account decides which partitions and QOS you may use. Check it before choosing a route:

```
sacctmgr -n show assoc user=$USER format=Account,Partition,QOS
myfairshare # read RealFairShare; a very low score means longer queue waits
```

02A / PARTITIONS

GPUs live in `htc`, `public`, `general`, `lightwork`, and `arm`. The deciding questions are how long the job runs, whether it may be preempted, and which accelerator it needs.

Partition	Wall limit	Hardware / best use
htc	4 h	Default for short CPU and GPU work; large accelerator pool.
public	7 days	Non-preemptable CPU/GPU jobs that need more than 4 h.
general	14 days	Privately-owned CPU/GPU nodes via <code>private</code> or <code>grp_*</code> QOS.
lightwork	1 day	Light development, compilation, bulk I/O, and <code>vscode</code> ; max 8 cores.
highmem	7 days	Memory-heavy CPU work, up to 2 TB; normal public use may have a lower cap.
arm	7 days	ARM/aarch64 nodes with Grace Hopper (gh200) GPUs.
fpga	7 days	FPGA, Vector Engine, and special accelerator workloads.

02B / QOS OVERLAYS

QOS	Wall cap	Use it for
public	Partition limit	Default, non-preemptable access to public resources.
debug	15 min	High-priority smoke tests; one running job and two submitted jobs per user.
private	Partition limit	Preemptible access to idle buy-in nodes; owners may cancel the job.
grp_*	Site/group limit	Your group's owned nodes, when your account provides the QOS.
long	14 days	Approved long batch jobs on <code>public</code> or <code>highmem</code> ; not for interactive jobs.
class	1 day	Course accounts; per-user CPU, memory, GPU, and job-count caps.

02C / ROUTING IN ONE GLANCE

Routing: up to 4 h -> `htc`; up to 15 min and urgent -> `-p htc -q debug`; more than 4 h -> `public`; more than 4 h and preemption is acceptable -> `-p general -q private`. Validate unusual combinations with `sbatch --test-only`.

03 / EVERYDAY SOLX WORKFLOW

```
solx init # create ~/.config/solx/config.toml
solx config edit # define job templates and [keep] paths
solx job start debug -n # preview the salloc command
solx job start debug # allocate; waits for the grant and prints the job ID
solx job jump # open a shell on the compute node
solx job time # show remaining wall-time
exit # leave the shell; the allocation keeps running
solx job stop # cancel when done; prompts before acting
```

job also accepts `jobs`; `job list` accepts `ls`; and `solx jump` is an alias for `solx job jump`.

04 / COMPLETE SOLX COMMAND MAP

Command	Purpose / important options
<code>solx init [-f]</code>	Write starter config; <code>-f</code> / <code>-y</code> overwrites.
<code>solx job list</code>	List your jobs (<code>squeue --me</code>).
<code>solx job start [TEMPLATE]</code>	Start an interactive allocation; <code>-n</code> , <code>--timeout</code> , and <code>--passthrough</code> .
<code>solx job jump [JOBID]</code>	Attach with <code>srunch --pty</code> ; <code>-q</code> hides nesting/selection notes.
<code>solx job time [JOBID]</code>	Print remaining time in D-HH:MM:SS.
<code>solx job stop [JOBID]</code>	Cancel a job; <code>-n</code> previews and <code>-y</code> skips confirmation.
<code>solx keep</code>	Renew flagged scratch files selected by <code>[keep]</code> ; see below.
<code>solx config show / edit</code>	Inspect resolved config or open it in <code>\$EDITOR</code> .
<code>solx completions bash zsh fish</code>	Emit a static shell-completion script.
<code>solx cheatsheet</code>	Print this quick reference as text.
<code>solx version / help</code>	Aliases of <code>--version</code> / <code>--help</code> .

05 / OUTPUT, TARGETING, AND SAFETY RULES

Situation	Rule
Human terminal	Data commands print aligned text.
Pipe or agent	Output auto-switches to JSON; <code>--json</code> forces it. Put <code>--json</code> before <code>job start</code> .
Destructive command	<code>job stop</code> and <code>keep show</code> a plan and prompt; <code>-n</code> previews, <code>-y</code> confirms.
Non-interactive session	A command that needs confirmation refuses instead of hanging.
Missing job ID	Inside an allocation, use <code>\$SLURM_JOB_ID</code> ; on login, <code>time / jump</code> pick the most recent job.
Multiple jobs on login	<code>stop</code> refuses to guess; pass the job ID.
Jump from another job	<code>solx job jump <JOBID></code> safely targets that allocation and warns about nesting; use <code>-q</code> to silence the note.

```
solx --json job list | jq '.[].job_id'
solx job start gpu --timeout 20m -- --mem=128G # last salloc flag wins
solx job stop 12345 -n # preview, never cancel
```

06 / SOLX KEEP - BOUNDED SCRATCH RENEWAL

`solx keep` touches only paths that are both in Sol's warning CSVs and matched by the config's [keep] include/exclude rules. It never walks all of `/scratch` blindly.

```
solx keep --dry-run -v      # inspect the full plan first
solx keep                  # execute with a confirmation prompt
solx keep --stage pending  # only the most urgent warning CSV
solx --json keep -n        # machine-readable counts and a capped sample
```

Other controls: `--stage all|inactive|over90|pending`, `--csv-dir DIR`, `-j N` workers, `-v`, `-n`, and `-y`. Run large renewals on the DTN (`ssh soldtn`), a compute node, or a short batch job - not on a throttled login node.

07 / SLURM BASICS

```
sbatch job.sh              # submit a batch script
squeue --me                # your jobs (human alias: myjobs)
scancel <jobid>            # cancel
scontrol show job <jobid>  # full detail
sbatch --test-only job.sh  # validate without submitting
interactive                # quick shell; defaults to htc/public, 1 core, 4 h
```

Minimal `#SBATCH` header (time format is D-HH:MM:SS):

```
#!/bin/bash
#SBATCH -p htc
#SBATCH -q public
#SBATCH -t 0-04:00:00
#SBATCH -c 8
#SBATCH --gres=gpu:a100:1
#SBATCH --mem=64G
#SBATCH -o slurm.%j.out
```

Start from `/packages/public/sol-sbatch-templates/templates/` when a supplied template fits. Use `solx job start` for interactive work and `sbatch` for unattended batch work.

08 / SOLX AND RAW SLURM EQUIVALENTS

	Raw Slurm
<code>solx job start [TEMPLATE]</code>	<code>salloc / interactive</code> with template flags.
<code>solx job jump [JOBID]</code>	<code>srun --jobid=ID --overlap --pty \$SHELL</code> .
<code>solx job list</code>	<code>squeue --me</code> .
<code>solx job time [JOBID]</code>	<code>squeue -h -j ID -o %L</code> .
<code>solx job stop -y ID</code>	<code>scancel ID</code> .

09 / JOB STUCK PENDING? DIAGNOSE BEFORE REROUTING

```
squeue --me -t PD -O "JobID,Reason:50,StartTime"  # reason and estimated start
scontrol show job <id>                          # all fields for one job
```

Reason	Response
Priority with low fairshare	Report the ETA and wait; resubmitting does not improve priority.
ReqNodeNotAvail	Check whether the requested node is reserved, drained, or down.
Resources	Right-size or reroute if another eligible partition starts sooner.

Only capacity-bound jobs benefit from rerouting. If a reroute helps, preserve the accrued queue priority with `scontrol update job <id> Partition=... QOS=...` instead of canceling and resubmitting.

10 / HUMAN WRAPPERS AND AGENT-PARSEABLE COMMANDS

You want	Human command	Parse this
Fairshare / priority	<code>myfairshare</code>	<code>myfairshare -> RealFairShare</code> .
Scratch quota	<code>-</code>	<code>beegfs-ctl --getquota --uid \$USER</code> .
Current jobs	<code>myjobs</code>	<code>squeue --me -O JobID,State,Reason</code> .
Pending start estimate	<code>thisjob ID</code>	<code>scontrol show job ID -> StartTime=</code> .
Finished-job efficiency	<code>seff ID</code>	<code>seff ID</code> .
Partition capacity	<code>showparts</code>	<code>sinfo -h -o "%P %a %l %D %t"</code> .
Free GPUs	<code>showgpu</code>	<code>sinfo -h -O "Partition,StateLong,Gres,GresUsed"</code> .

`my*` / `show*` tools are colorized for people. Agents should prefer native Slurm fields or `solx --json`; free GPUs equal `Gres` minus `GresUsed`.

11 / REACH A COMPUTE-NODE SERVICE FROM YOUR LAPTOP

```
# On a Sol login node: register a VS Code tunnel on lightwork.
vscode

# On your laptop: forward Jupyter on compute node $NODE, port 8888.
ssh -N -L 8888:localhost:8888 -J $USER@login.sol.rc.asu.edu $USER@$NODE
```

Get `$NODE` from the `NODELIST` column in `squeue --me`. Bind services to `localhost`, never `0.0.0.0`, on shared nodes.

12 / STORAGE AND HEAVY I/O

Path	Use	Policy
<code>/scratch/\$USER</code>	Datasets, caches, checkpoints, outputs.	Temporary, not backed up; inactive files are purged.
<code>/home/\$USER</code>	Code, config, and small user installs.	Persistent, backed up, small quota.

```
export HF_HOME=/scratch/$USER/.cache/huggingface
export UV_CACHE_DIR=/scratch/$USER/.cache/uv
```

Use the DTN (`ssh soldtn`) for bulk transfer and metadata-heavy I/O. Use a compute node or batch job for compute. Keep heavy work off login nodes.

13 / FIVE RULES TO REMEMBER

Rule	Default
Preview changes	Use <code>-n / --dry-run</code> before <code>job stop</code> , <code>keep</code> , or allocation changes.
Parse output	Use <code>solx --json ...</code> or native Slurm fields; do not scrape colorized wrappers.
Route short jobs	Use <code>htc</code> for work up to 4 h, including GPU jobs.
Protect the login node	Move compute and metadata-heavy I/O to a compute node, batch job, or DTN.
Preserve queue priority	Diagnose <code>PENDING</code> ; update a viable route in place instead of resubmitting.